

《面向对象程序设计实践》

课程论文

题目：	思维导图绘制工具
专业：	计算机科学与技术/软件工程
班级：	2024 级软件工程 5 班
小组成员 1：	202425220512 蓝淑琪
小组成员 2：	202425220504 陈依婕
小组成员 3：	202414510225 许婧言
指导老师：	彭红星
提交时间：	2026 年 5 月 20 日

华南农业大学 数学与信息学院 软件学院

目录

1 系统分析.....	3
1.1 问题描述.....	3
1.2 系统功能分析.....	3
1.3 开发平台及工具介绍.....	4
2 系统设计.....	4
2.1 系统总体结构设计.....	4
2.2 系统各个类及类之间关系设计.....	5
2.3 数据存储的设计.....	6
2.4 界面设计.....	7
3 系统实现.....	8
4 系统测试.....	13
4.1 模块测试.....	13
4.2 系统测试.....	15
5 系统运行界面.....	16
6 总结.....	17

1 系统分析

1.1 问题描述

在现代知识管理和创意激发过程中，思维导图（Mind Map）是一种被广泛采用的工具，它通过可视化的方式梳理信息、激发联想。然而，市场上的许多思维导图软件要么是收费的，要么是基于浏览器的，且对私有部署和离线使用的支持不一。

本项目旨在开发一个基于 **Java Swing** 的原生、轻量级、面向对象程序设计的思维导图桌面应用程序。该程序的核心目标是提供一个灵活、可扩展的平台，让用户可以直观地创建、编辑、管理和保存自定义的思维导图。系统需要解决的关键问题包括：

（1）**数据模型构建**：如何建立一个高效、面向对象的树状结构模型，以准确表达思维导图的层级和关联。

（2）**界面交互**：如何在一个可视化的画布上实现复杂的用户交互，如节点的动态添加、删除、编辑、拖拽移动、形状自定义，以及画布的平移和缩放。

（3）**自动化布局**：如何设计布局算法，自动排列复杂的节点网络，使其保持美观和易读，防止节点重叠。

（4）**状态管理**：如何实现操作的历史记录管理，提供撤销（Undo）和重做（Redo）功能，以保证用户在编辑过程中的数据安全。

（5）**持久化与导出**：如何将内存中的数据模型保存为文件，以及将思维导图导出为高质量的图片，便于分享。

1.2 系统功能分析

（1）**思维导图核心模型模块（MindMap）**：负责管理思维导图的整体结构，包括根节点、节点间的层级关系、当前选中的节点等。它包含所有节点的操作方法（添加子节点、删除节点、查找节点等）。

（2）**节点定义与绘制模块（MindMapNode, NodeShape）**：定义了思维导图中的最小单位——节点，包含节点的文本内容、几何属性（位置、大小）、形状类型（矩形、椭圆、菱形、圆柱体等）、边框和填充颜色。同时，它负责在画布上绘制自身和与子节点之间的连接线。

（3）**可视化画布模块（MindMapPanel）**：是用户交互的核心区域，负责管理画布的缩放、平移、鼠标事件监听（拖拽节点、双击编辑文本、选择节点等）。它还持有并绘制整个思维导图的模型。

（4）**主程序界面模块（MindMapFrame, TreePanel）**：提供了应用程序的主窗口框架、工具栏（包含添加、删除节点、改变形状、撤销、重做、保存、导出等按钮），以及一个用于展示节点层级结构的目录树面板（TreePanel）。

（5）**自动化布局管理模块（LayoutManager）**：实现对思维导图的自动排版，支持将节点以根节点为中心，向左侧、右侧、或者自动平衡排列，防止节点重叠，计算节点的动态尺寸。

（6）**操作历史与状态管理模块（HistoryManager）**：记录用户的每一个编辑操作，维护撤销（Undo）和重做（Redo）两个堆栈。它通过对 MindMap 对象进行深拷贝（利用 Java 序列化）来保存完整的状态。

（7）**文件管理与导出模块（FileManager, ImageExporter）**：负责将 MindMap 模型对象序列化为本地文件（.dt 格式），以便后续加载；同时支持将可视化的思维导图导出为 PNG 或 JPG 图片。

1.3 开发平台及工具介绍

- (1) **开发语言**: Java (面向对象程序设计)
- (2) **图形用户界面 (GUI) 库**: Java Swing (原生 Java GUI 工具包)
- (3) **操作系统 (开发与运行)**: 跨平台, 基于 Java 虚拟机 (JVM) 运行, 适用于 Windows (如截图所示)、macOS 和 Linux。
- (4) **构建与运行脚本**: 利用 `Windows.bat` 批处理文件进行一键式编译、打包和运行管理。
- (5) **核心特性**: Java 序列化 (用于状态保存和文件存储)、对象克隆 (深拷贝用于历史记录)、图形学绘制 (Canvas, Graphics2D)。

2 系统设计

2.1 系统总体结构设计

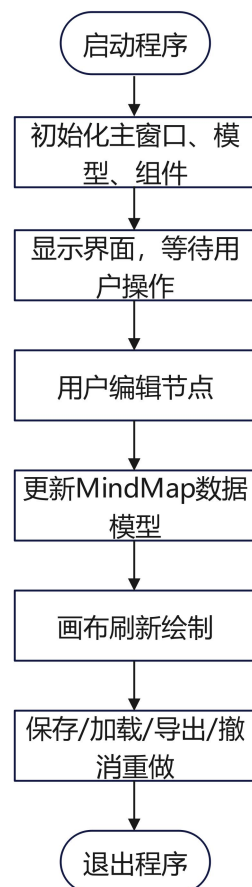
本思维导图系统采用面向对象的模块化分层架构, 整体分为**数据模型层**、**界面视图层**、**业务逻辑层**、**工具与持久化层**四个部分。

(1) **数据模型层**: 负责思维导图的核心数据结构, 包括 `MindMap` 整体模型与 `MindMapNode` 节点模型, 存储节点的文本、位置、形状、颜色、父子关系等信息, 是整个系统的数据基础。

(2) **界面视图层**: 基于 Java Swing 实现, 包括主窗口 `MindMapFrame`、画布面板 `MindMapPanel`、树形结构面板 `TreePanel`, 负责节点绘制、用户交互、菜单与工具栏显示, 提供可视化操作界面。

(3) **业务逻辑层**: 包含自动布局 `LayoutManager`、历史记录 `HistoryManager`, 处理节点自动排版、撤销等核心功能, 保证思维导图结构清晰、编辑操作可回溯。

(4) **工具与持久化层**: 提供文件保存及加载 `FileManager`、图片导出 `ImageExporter` 以及枚举类型 `LayoutType`、`ConnectorType`、`NodeShape`, 支持数据持久化、样式化控制与扩展



2.2 系统各个类及类之间关系设计

本系统共包含 10 个核心类与枚举，类之间以关联、依赖、聚合关系组织。

(1) **MindMap (思维导图模型类)**：管理根节点、所有节点列表、当前选中节点，提供添加、删除、查找、遍历节点的方法，是整个系统的数据中心。

(2) **MindMapNode (节点类)**：思维导图最小单位，存储内容、坐标、父节点、子节点列表、形状、颜色，具备添加、删除子节点，更新内容等功能。

(3) **MindMapFrame (主窗口类)**：系统入口类，继承 JFrame，初始化菜单、工具栏、画布、树形面板，协调各模块工作。

(4) **MindMapPanel (画布面板类)**：继承 JPanel，负责节点与连接线绘制，处理鼠标点击、拖拽、缩放、平移等交互事件。

(5) **TreePanel (树形结构面板)**：以树形结构展示节点层级，与画布双向同步，方便快速定位与编辑。

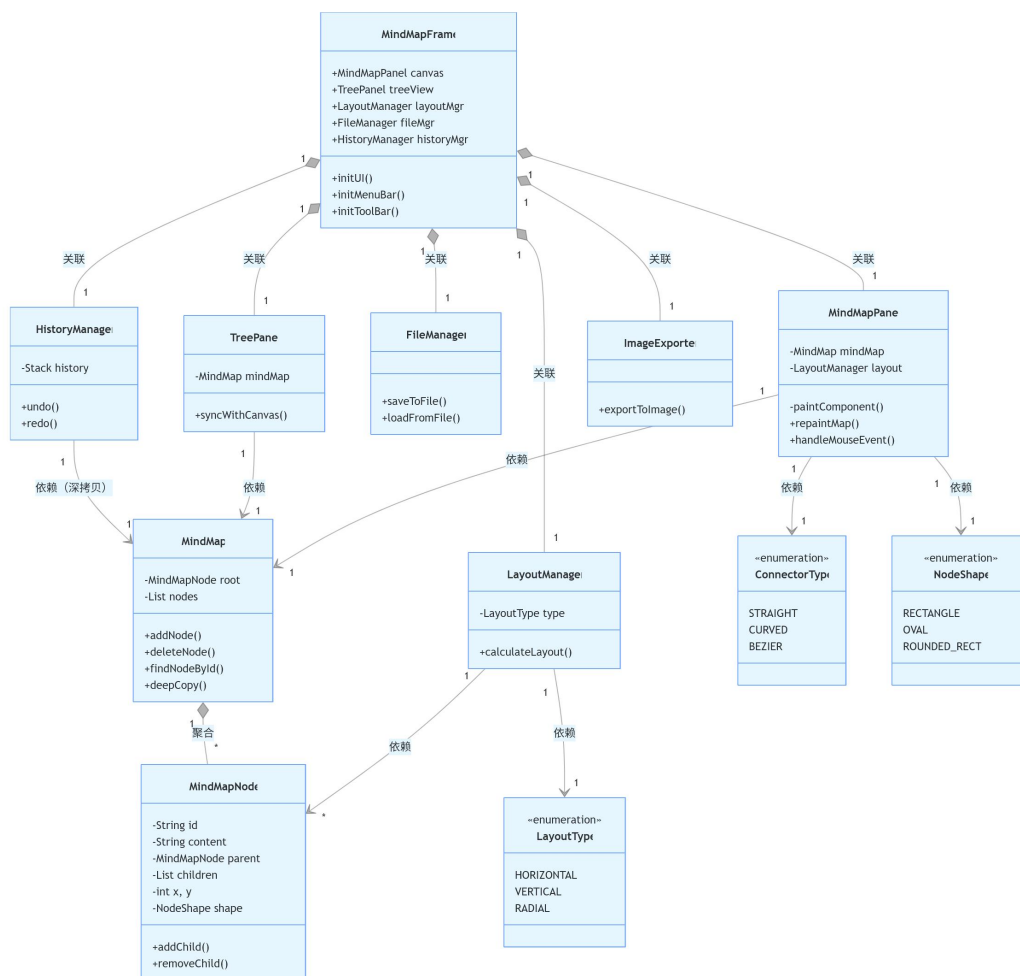
(6) **LayoutManager (布局管理器)**：根据布局类型（自动、左、右）递归计算节点位置，防止重叠，实现自动排版。

(7) **HistoryManager (历史记录管理器)**：使用深拷贝与栈结构保存操作状态，实现撤销（Undo）、重做（Redo）功能。

(8) **FileManager (文件管理器)**：基于 Java 序列化实现 .dt 格式文件的保存与加载。

(9) **ImageExporter (图片导出器)**：将画布内容导出为 PNG/JPG 图片。

(10) **LayoutType、ConnectorType、NodeShape (枚举类)**：定义布局方式、连接线样式、节点形状、统一系统常量规范。

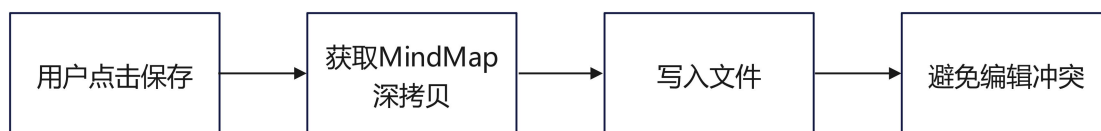


系统核心类 UML 图

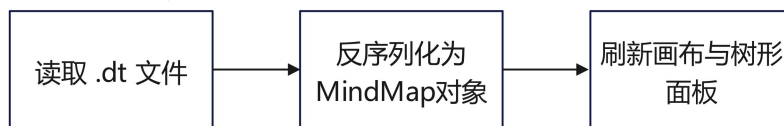
2.3 数据存储的设计

系统采用 Java 序列化实现数据持久化，保存格式为自定义后缀.dt 文件，能够完整存储思维导图的所有结构与样式。存储内容包括**根节点信息**，所有节点的**文本、位置、颜色、形状、父子关系**，布局类型、连接线样式、画布状态等。

(1) 保存流程：



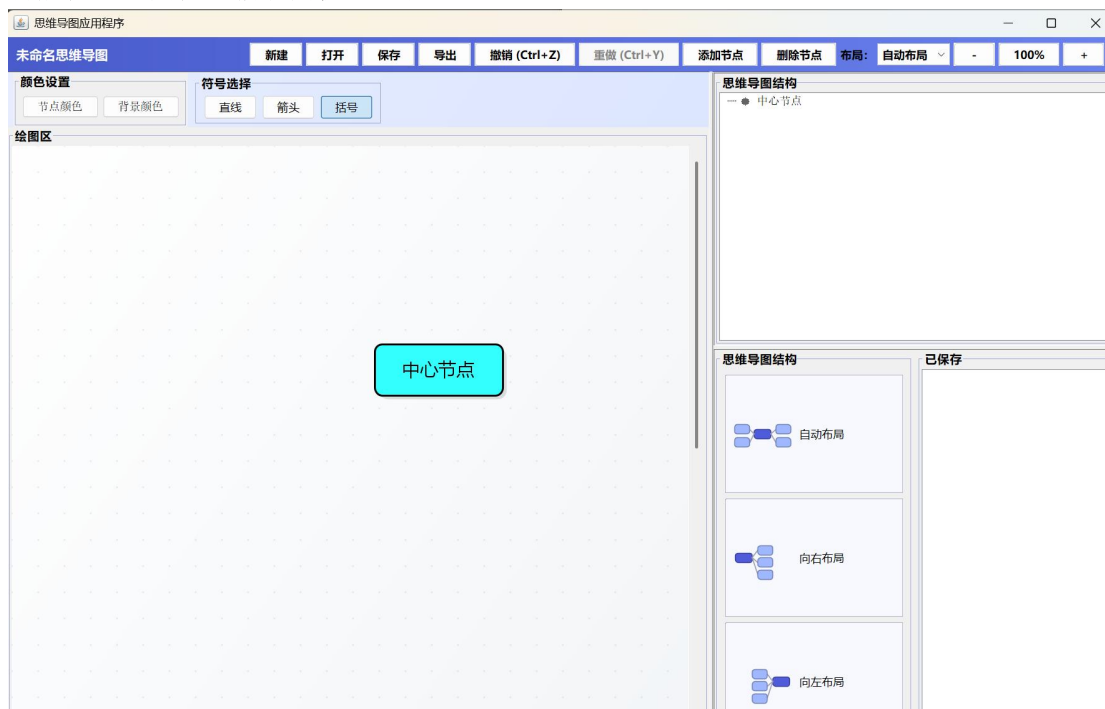
(2) 加载流程：



(3) 异常处理：捕获文件不存在、格式错误、读写权限异常，给出提示，保证程序稳定。

2.4 界面设计

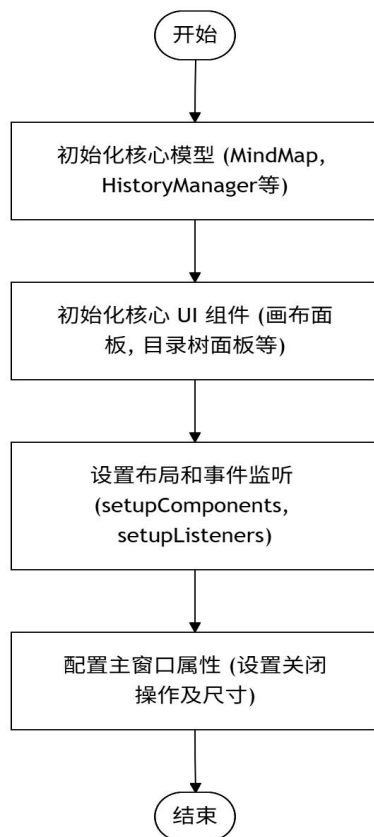
采用 Java Swing 实现图形用户界面，整体布局清晰、交互友好，分为菜单栏、工具栏、功能栏、绘图栏、辅助面板五大模块，界面截图如下：



界面支持多种交互功能：

- (1) **节点操作交互**：画布中默认显示中心节点，支持双击节点编辑文本内容；点击【**删除节点**】可删除选中节点（或选中节点及其所有子节点）；拖拽节点可自由调整位置，系统自动更新连接线走向，保持节点间的关联关系。
- (2) **布局与样式交互**：顶部工具栏的【**布局**】下拉菜单，支持切换自动布局、左右向布局，一键优化节点分布；左侧功能区支持自定义节点颜色、背景颜色和连接线样式，实现个性化思维导图效果。
- (3) **视图联动交互**：绘图区与右侧【**思维导图结构**】树状面板双向同步；布局预览面板点击不同布局选项，绘图区实时更新节点排版效果。
- (4) **文件与编辑交互**：支持新建空白思维导图、打开本地文件、保存当前编辑内容、导出为图片格式；提供撤销重做功能，支持多步操作回退。

3 系统实现



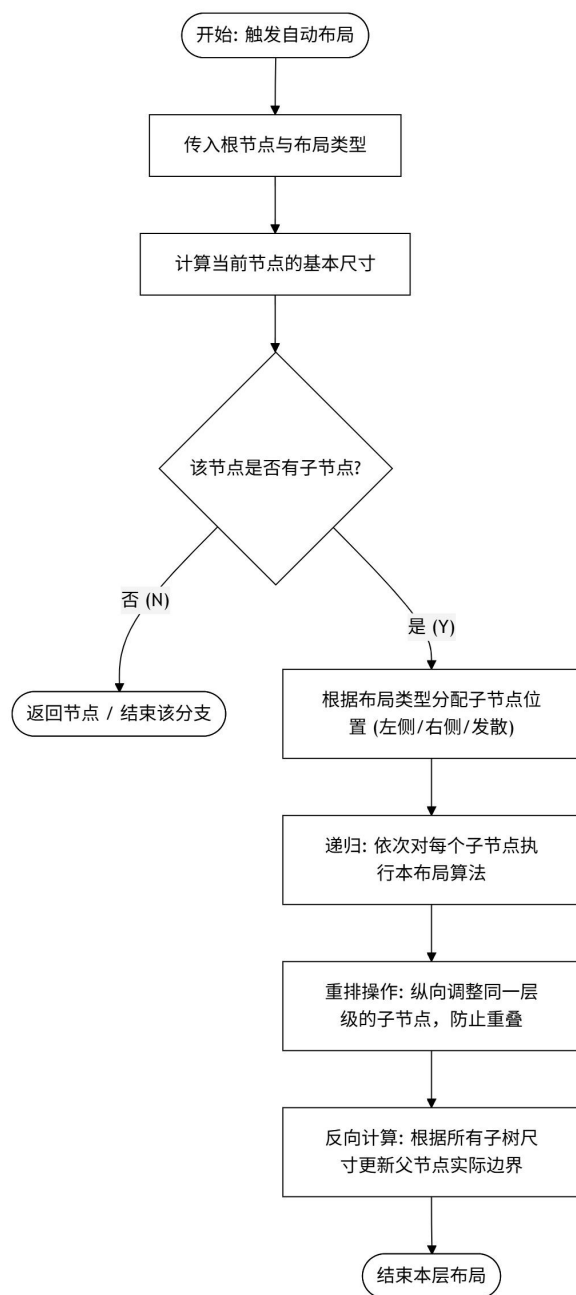
// MindMapFrame.java

```
public MindMapFrame() {  
    // 1. 初始化核心模型  
    mindMap = new MindMap();  
    historyManager = new HistoryManager(mindMap);  
    fileManager = new FileManager(this);  
    imageExporter = new ImageExporter(this);  
  
    // 2. 初始化核心 UI 组件  
    // 创建画布面板 (左侧)  
    mindMapPanel = new MindMapPanel(mindMap, historyManager);  
    // 创建目录树面板 (右侧)  
    treePanel = new TreePanel(mindMap);  
    // 创建预览面板 (右侧底部)  
    previewPanel = new PreviewPanel(mindMapPanel);  
  
    // 3. 设置布局 and 事件监听  
    setupComponents(); // 设置工具栏、布局、监听器  
    setupListeners();  // 设置核心监听器
```

```

setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setSize(1024, 768);
}

```



// LayoutManager.java (核心布局逻辑)

```

private void calculateLayoutRecursive(MindMapNode node, LayoutType type) {
    if (node == null) return;

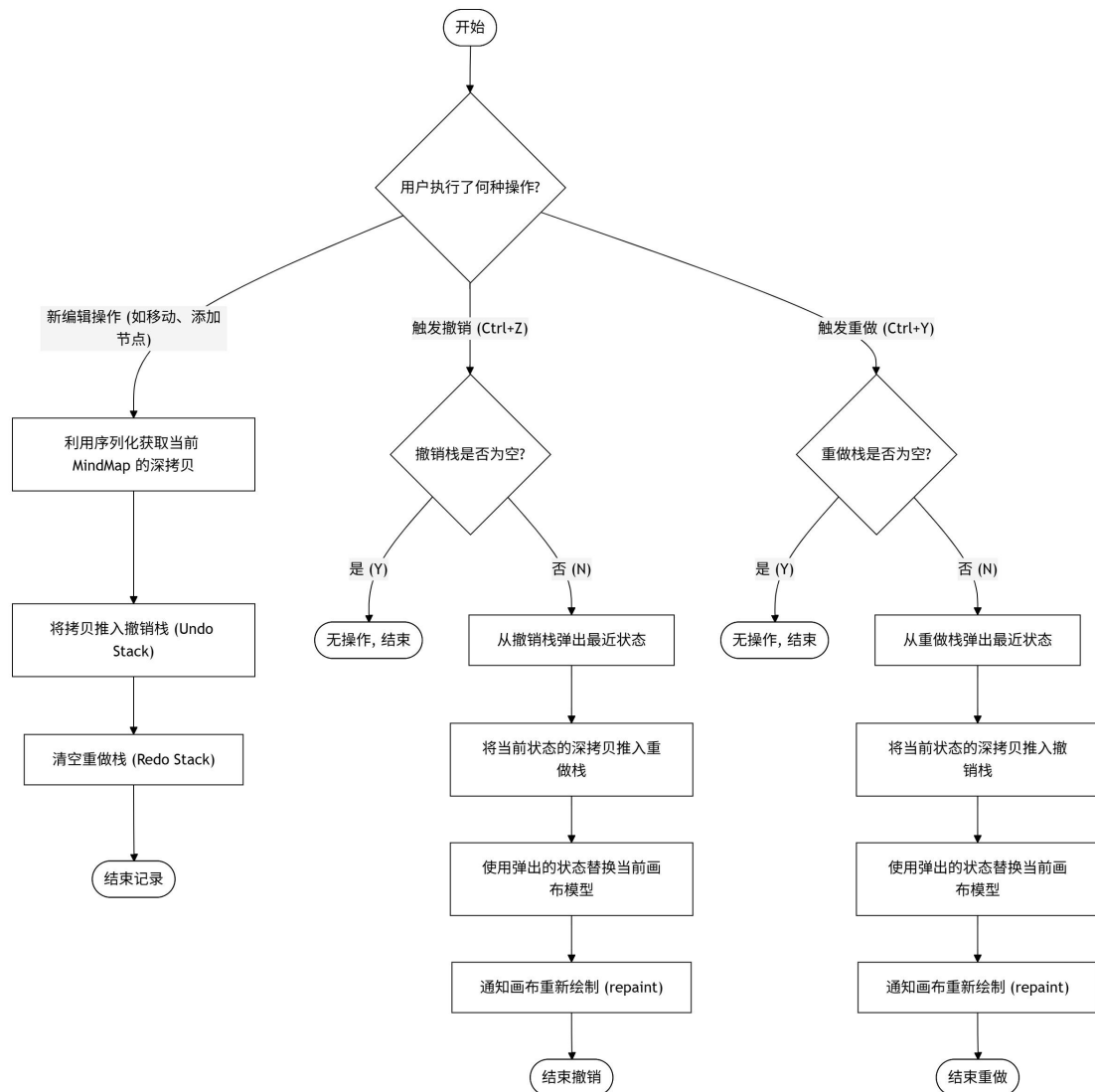
    // 1. 计算当前节点的基本尺寸
    calculateNodeDimension(node);
    // 2. 如果没有子节点, 直接返回
    if (node.getChildren().isEmpty()) {

```

```

    return;
}
// 3. 处理子节点: 分配子节点到左侧、右侧 (基于 LayoutType)
distributeChildren(node, type);
// 4. 递归计算子节点的布局 (深度优先)
for (MindMapNode child : node.getChildren()) {
    calculateLayoutRecursive(child, type);
}
// 5. 子节点布局完成后, 反向调整父节点
// 调整纵向位置以防止重叠 (核心算法)
rearrangeChildrenVertically(node);
// 计算父节点根据所有子树合并后的实际边界
calculateParentNodeBounds(node);
}

```



```

// HistoryManager.java (核心逻辑)
public void undo() {
    if (undoStack.isEmpty()) {

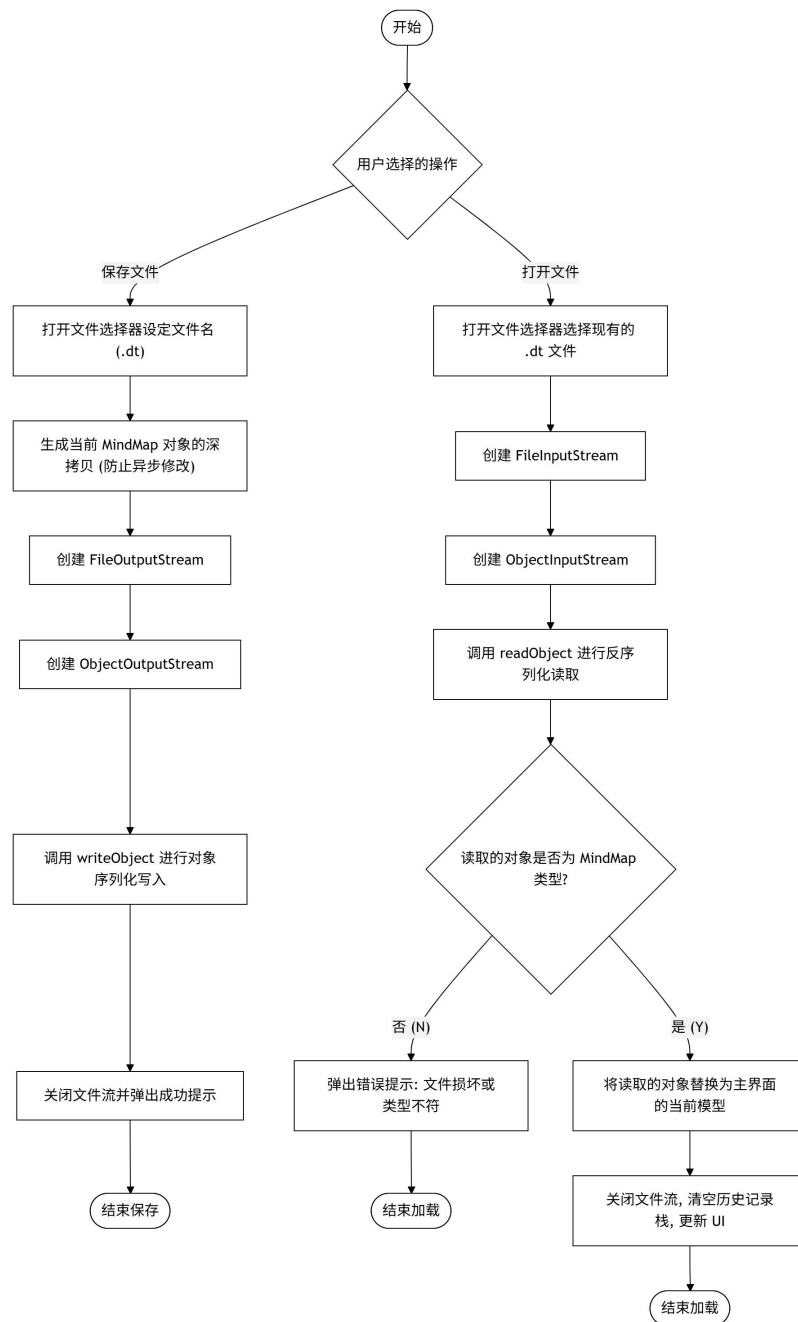
```

```

        return; // 没有可撤销的操作
    }
    // 1. 从撤销栈弹出最近的状态
    MindMap undoneMap = undoStack.pop();
    // 2. 将当前状态的深拷贝推入重做栈
    redoStack.push(mindMap.cloneDeep());
    // 3. 替换当前的数据模型
    this.mindMap = undoneMap;
    // 4. 刷新界面
    frame.updateMindMap(mindMap);
}

public void redo() {
    if (redoStack.isEmpty()) {
        return; // 没有可重做的操作
    }
    // 1. 从重做栈弹出状态
    MindMap redoneMap = redoStack.pop();
    // 2. 将当前状态的深拷贝推入撤销栈
    undoStack.push(mindMap.cloneDeep());
    // 3. 替换数据模型
    this.mindMap = redoneMap;
    // 4. 刷新界面
    frame.updateMindMap(mindMap);
}

```



```

// FileManager.java (核心序列化逻辑)
public void save(File file) {
    if (file == null) return;
    // 1. 在保存前, 获取当前模型的一个完整深拷贝
    // 防止在序列化过程中用户编辑导致数据不一致
    MindMap copyOfMap = mindMap.cloneDeep()
    // 2. 使用 Try-With-Resources 自动管理流
    try (FileOutputStream fos = new FileOutputStream(file);
        ObjectOutputStream oos = new ObjectOutputStream(fos)) {
        // 3. 将整个模型对象序列化写入文件
        oos.writeObject(copyOfMap);
    }
}

```

```

        // 提示保存成功
        System.out.println("Saved mind map to " + file.getAbsolutePath())
    } catch (IOException e) {
        // 4. 异常处理：例如文件权限不足
        e.printStackTrace();
    }
}
}

```

4 系统测试

4.1 模块测试

(1) LayoutType.java（布局类型枚举）

测试目的：验证布局枚举可正常获取类型及对应描述

测试步骤：

调用 AUTO、LEFT、RIGHT 三种枚举值

查看各枚举值的描述文本

预期结果：三种布局类型正常识别，描述文字显示正确

实际结果：三种布局类型正常识别并清晰展示

(2) ConnectorType.java（连接符枚举）

测试模块：ConnectorType.java

测试目的：验证连接符枚举可正常识别并获取对应样式描述

测试步骤：

调用 LINE、ARROW、BRACKET 三种枚举值

查看各连接符对应的显示名称

预期结果：三种连接符类型正常识别，样式描述显示正确

实际结果：三种连线样式都能正常显示，切换后节点间连线变化正确

系统测试：系统功能的完整测试

说明：测试步骤为

<1>设计测试用例：包括输入数据，预期结果。

<2>实际运行程序得到：运行结果

<3>分析预期结果与运行结果的异同得到测试结论。

(3) FileManager.java（文件管理）

测试模块：FileManager.java

测试目的：验证思维导图文件的保存与加载功能正常

测试步骤：

创建思维导图，编辑节点内容后保存为.dt 文件

关闭当前导图，重新打开保存的文件

预期结果：文件保存成功，重新打开后导图内容完整无缺失

实际结果：文件保存成功，并且可以重复保存

(4) HistoryManager.java（撤销重做）

测试模块：HistoryManager.java

测试目的：验证操作的撤销与重做功能可正常使用

测试步骤：

执行添加、修改节点的操作

分别点击撤销、重做按钮

预期结果：撤销可回退上一步操作，重做可恢复撤销的操作

实际结果：撤销可回退上一步操作，重做可恢复撤销的操作，并且重做按钮有明暗显示

(5) ImageExporter.java (图片导出)

测试模块：ImageExporter.java

测试目的：验证思维导图可正常导出为图片文件

测试步骤：

完成思维导图的编辑与布局调整

选择 PNG/JPG 格式，执行导出操作

预期结果：成功生成图片，画面内容与当前导图完全一致

实际结果：成功导出，并且内容相同

(6) LayoutManager.java (布局管理)

测试模块：LayoutManager.java

测试目的：验证不同布局可正常对节点进行排列

测试步骤：

创建包含多层子节点的思维导图

切换自动、左、右三种布局模式

预期结果：节点按所选布局整齐排列，无重叠、错位现象

实际结果：可以选择任意节点进行布局，反应灵敏且正确

(7) MindMapNode.java (节点数据模块)

测试模块：MindMapNode.java

测试目的：验证单个节点的基础属性可正常设置和显示

测试步骤：

创建节点，设置文本、位置、颜色、形状

查看节点显示效果及各项属性

预期结果：节点文本、位置、颜色、形状均正常生效，显示正确

实际结果：节点文本、位置、颜色、形状均正常生效

(8) 测试模块：MindMap.java (思维导图数据管理模块)

测试模块：MindMap.java

测试目的：验证整个思维导图能正常管理节点层级、增删及选中状态

测试步骤：

对导图节点进行添加、删除、选中操作

查看节点层级关系及选中状态变化

预期结果：节点增删生效，层级关系正确，选中状态切换正常

实际结果：节点增删生效，层级关系正确，选中状态切换正常

(9) MindMapPanel.java (画布绘制与交互模块)

测试模块：MindMapPanel.java

测试目的：验证画布可正常绘制导图、响应拖拽和缩放等交互操作

测试步骤：

查看画布上节点、连线的绘制效果

拖拽节点、缩放画布、切换连线样式

预期结果：绘制清晰，拖拽、缩放流畅，连线样式切换生效

实际结果：绘制清晰，拖拽、缩放流畅，连线样式切换生效

(10) 测试模块: MindMapFrame.java (主窗口界面模块)

测试模块: MindMapFrame.java

测试目的: 验证主窗口整体显示正常, 各功能按钮可正常响应

测试步骤:

启动程序, 查看主窗口及菜单栏、工具栏显示

点击新建、添加、删除、保存等功能按钮

预期结果: 窗口显示完整美观, 各按钮点击后对应功能正常触发

实际结果: 窗口显示完整美观

4.2 系统测试

测试用例 1: 思维导图完整创建与存储流程

<1> 设计测试用例

输入 / 操作数据:

- ①启动程序, 新建空白思维导图;
- ②修改中心节点文本为“学习计划”;
- ③添加 3 个子节点, 分别命名“数据库”“面向对象程序设计”“计算机网络”;
- ④切换为自动布局, 调整节点位置;
- ⑤保存导图为学习计划.dt, 关闭程序;
- ⑥重新启动程序, 打开学习计划.dt 文件。

预期结果:

新建后自动生成中心节点;

节点文本修改、子节点添加成功;

布局切换后节点排列整齐;

文件保存成功, 重新打开后导图内容、布局、文本与保存前完全一致。

<2> 实际运行结果

程序正常启动, 可新建导图; 中心节点文本修改、子节点添加均成功; 切换自动布局后节点排列整齐; 文件保存无报错, 关闭后重新打开, 导图所有内容完整无缺失, 布局与文本均正确。

<3> 测试结论

预期结果与运行结果一致, 思维导图新建、编辑、布局、保存、打开的完整流程功能正常, 数据存储稳定。

测试用例 2: 综合编辑与导出功能流程

<1> 设计测试用例

输入 / 操作数据:

- ①打开已有思维导图;
- ②编辑节点文本、修改节点颜色和形状;
- ③删除 1 个多余节点, 拖拽调整节点位置;
- ④执行 2 次撤销、1 次重做操作;
- ⑤切换连线样式为箭头, 导出导图为 PNG 图片。

预期结果:

- ①节点文本、颜色、形状修改生效;
- ②节点删除、拖拽位置调整正常;

③撤销可回退操作，重做可恢复操作；

④连线样式切换成功，导出的 PNG 图片与当前导图显示一致。

<2> 实际运行结果

节点文本、样式修改成功；节点删除、拖拽流畅；撤销、重做操作可正常回退和恢复；连线样式切换后画布实时更新；PNG 图片导出成功，画面清晰，内容与导图完全匹配。

<3> 测试结论

预期结果与运行结果一致，思维导图综合编辑、撤销重做、样式切换、图片导出功能联动正常，交互响应流畅。

测试用例 3：基础功能稳定性测试

<1> 设计测试用例

输入 / 操作数据：

①连续多次新建、切换思维导图；

②频繁添加、删除节点，反复切换布局；

③多次保存、打开文件，导出不同格式图片。

预期结果：

①程序无卡顿、闪退；

②所有操作无报错，功能响应正常；

③数据无丢失、无错乱，界面显示稳定。

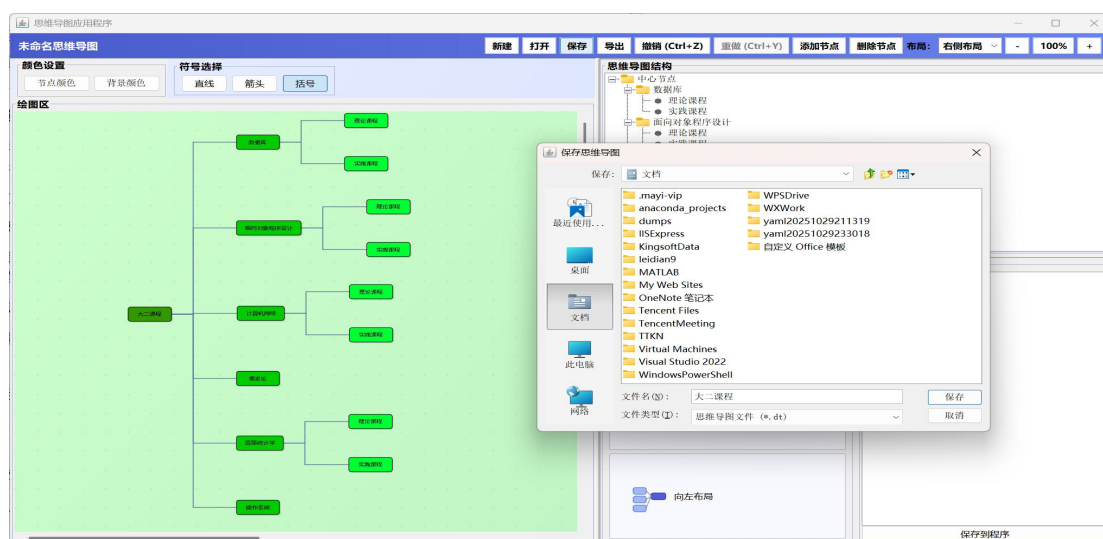
<2> 实际运行结果

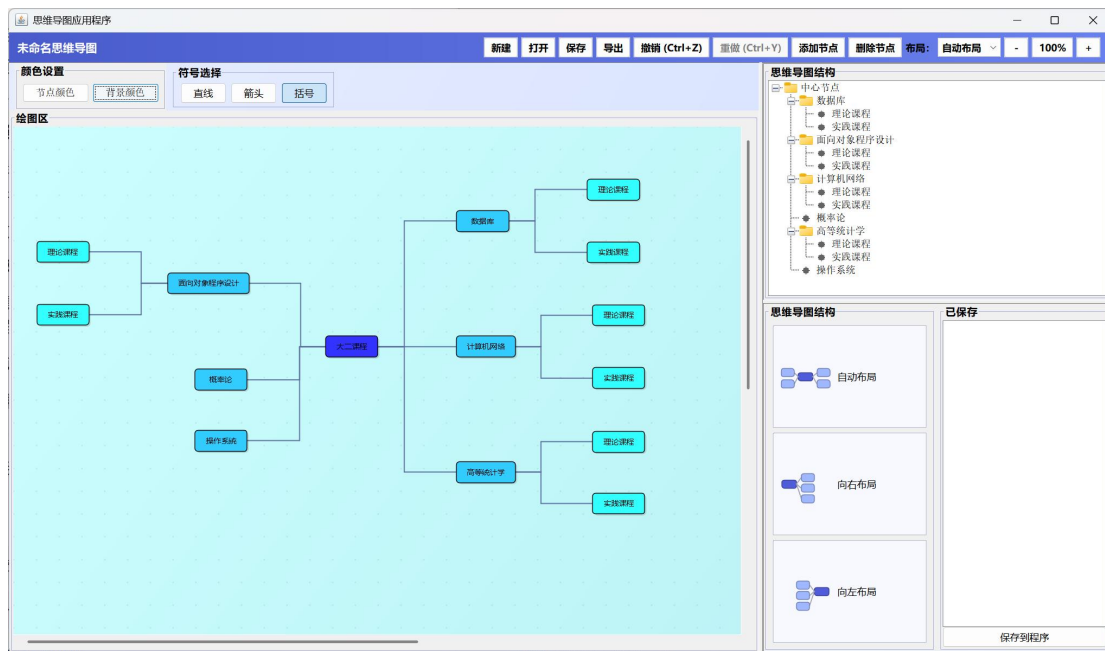
多次重复操作后，程序运行流畅，无卡顿、闪退或报错；节点、布局、文件数据均无丢失错乱，界面显示正常稳定。

<3> 测试结论

预期结果与运行结果一致，系统稳定性和容错性良好，频繁操作下核心功能始终正常。

5 系统运行界面





6 总结

成员 1:

在本次思维导图课程设计中，我主要负责项目的细节优化、体验打磨与联调支持工作。

开发过程中，我针对界面交互的流畅性进行了优化，参与了画布节点选中高亮、连接线样式适配、布局切换效果等细节实现，让用户操作反馈更清晰直观。同时，我配合团队完成了模块间的接口联调，协助解决了节点数据更新与视图同步、文件保存加载后的界面刷新等问题，保障了各功能模块的协同工作。后期我参与了程序的整体测试，整理了常见问题与修复方案，为项目的稳定性提供了支持。

通过这次课程设计，我深刻体会到了软件工程中细节优化与团队协作的重要性，也熟练掌握了 Java Swing 界面组件的调试与联调方法，积累了项目协同开发的宝贵经验。

成员 2:

这次面向对象课程设计，我主要负责思维导图程序的整体规划、雏形搭建，还完成了系统测试撰写和运行界面整理工作。前期我梳理了程序要实现的功能，划分好各个模块，用 Java Swing 搭好了程序基础框架，确定核心类的分工，让项目能顺利开始开发。后期我写了模块测试和系统测试用例，挨个验证功能是否正常，同时整理了程序运行界面的截图和说明。通过这次设计，我更熟练地掌握了 Java 编程和面向对象的思路，也明白了前期规划的重要性，还学会了写测试文档。过程中也发现自己实践能力还有欠缺，这次经历让我收获很多，也积累了实际开发的经验。

成员 3:

作为项目 UI 架构和文件系统的负责人，我的工作重点在于提升软件的工程完整度与用户体验（UX）。在 MindMapFrame 的设计中，我使用了 JSplitPane 和多种 Swing 布局管理器，打造了左侧绘图、右侧大纲与模板库联动的专业级界面。特别是实现了‘一键生成模板’功能，极大地降低了用户的使用门槛。

在开发 FileManager 和 ImageExporter 时，我处理了大量的异常捕获与边界计算。比如在导出图片时，需要递归遍历所有节点以计算出最大包围盒（Bounding Box），并在内存中通过 BufferedImage 绘制出一张带有安全边距的高清图片。在整个项目的联调阶段，我负

责协调画布渲染与排版算法的接口对接,这让我充分意识到了团队协作中接口定义和模块化规范的重要性。看着一个个零散的代码文件最终组装成一个可以保存、打开、导出成品的完整软件,我获得了极大的成就感。